Practical File

Database Management System

Submitted By –

Gaurav

Roll no -24570019

Course – BSc(H) Computer Science

Sem – 4th

Submitted to –

Ms. Sheetal Singh

(Assistant Professor)

Department of Computer Science

Ramanujan College, DU

I. Create and use the following student-society database schema for a college to answer the given (sample) queries using the standalone SQL editor.

STUDENT	**Roll No**	**StudentName**	**Course**	**DOB**
	Char(6)	Varchar(20)	Varchar(10)	Date

SOCIETY	**SocID**	**SocName**	**MentorName**	**TotalSeats**
	Char(6)	Varchar(20)	Varchar(15)	Unsigned int

ENROLLMENT	**Roll No**	**SID**	**DateOfEnrollment**
	Char(6)	Char(6)	Date

Here Rollno (ENROLLMENT) and SID (ENROLLMENT) are foreign keys.

```sql
drop database student_society;
create database student_society;
show databases;
use student_society;
create table  Student(ROLLNO char(6) primary key ,Student_name varchar(20) ,Course varchar(10),DOB date);
show tables;

insert into Student
values
('X1','Lakhan','ComputerSc','2004-11-13'),
('X2','Ram','chemistry','2005-09-03'),
('X9','Ajay','Physics','2003-03-11'),
('Z9','Aman','Maths','2006-05-10'),
('X4', 'Raju', 'Biology','2001-01-10');

select *from student;
desc Student;

create table society(SocID char(6) primary key, SocName varchar(20), MentorName varchar(15), TotalSeats int unsigned);
show tables;

insert into society
values
```

```sql
insert into society
values
    ('s1', 'Debating', 'Aman Gupta',25),
    ('s2', 'NSS', 'Raj',20),
    ('s3','Dancing','Aryan',13),
    ('s4', 'Sashakt', 'Bhatia',10),
    ('s5','QuickSort', 'Ms.Sheetal',15);
select *from society;

create table enrollment(ROLLNO char(6), SID char(6), DateofEnrollment date, foreign key(ROLLNO) references Student(ROLLNO), foreign key(SID) references society(SocID));
show tables;

insert  into enrollment
values
    ('X1','s1','2019-01-02'),
    ('X2','s2', '2019-03-15'),
    ('X9','s3','2018-11-19'),
    ('Z9','s4','2020-01-11'),
    ('X4','s5','2021-03-27');
desc enrollment;
select*from enrollment;
```

```sql
INSERT INTO enrollment VALUES ('X1','s1','2022-01-01');
INSERT INTO enrollment VALUES ('X2','s1','2022-01-02');
INSERT INTO enrollment VALUES ('X9','s1','2022-01-03');
INSERT INTO enrollment VALUES ('Z9','s1','2022-01-04');
INSERT INTO enrollment VALUES ('X4','s1','2022-01-05');
```

Queries –

1 - Retrieve names of students enrolled in any society.

```sql
-- Queries
-- 1
SELECT DISTINCT s.Student_name
FROM Student s
JOIN enrollment e
ON s.ROLLNO = e.ROLLNO;
```

Result Grid | Filter Rows:

Student_name
Lakhan
Ram
Raju
Ajay
Aman

2 - Retrieve all society names.

```sql
select SocName
from Society;
```

Result Grid | Filter Rows:

SocName
Debating
NSS
Dancing
Sashakt
QuickSort

3 - Retrieve students' names starting with the letter ‘A’.

```
-- 3
select distinct Student_name
from Student
where Student_name like 'A%';
```

Result Grid | Filter Rows:

Student_name
Ajay
Aman

4 - Retrieve students' details studying in courses ‘computer science’ or ‘chemistry’.

```
-- 4
select *
from student
where Course = 'ComputerSc' or Course = 'Chemistry';
```

Result Grid | Filter Rows: | Edit: | Expo

ROLLNO	Student_name	Course	DOB	MobileNo
X1	Lakhan	ComputerSc	2004-11-13	99999999
X2	Ram	chemistry	2005-09-03	99999999
NULL	NULL	NULL	NULL	NULL

5 - Retrieve students’ names whose roll no either starts with ‘X’ or ‘Z’ and ends with ‘9’

```
-- 5
select Student_name
from student
where (ROLLNO like 'X%9' or  ROLLNO like 'Z%9');
```

Result Grid | Filter Rows:

Student_name
Ajay
Aman

6 - Find society details with more than N TotalSeats where N is to be input by the user

```
select *
from society
where TotalSeats > 15;
```

Result Grid | Filter Rows: | Edit:

SocID	SocName	MentorName	TotalSeats
s1	Debating	Aman Gupta	25
s2	NSS	Raj	20
NULL	NULL	NULL	NULL

7 - Update society table for the mentor name of a specific society

```
-- 7
update society
set MentorName  = 'Gaurav'
where SocID = 's3';
select *from society;
```

Result Grid | Filter Rows: | Edit:

SocID	SocName	MentorName	TotalSeats
s1	Debating	Aman Gupta	25
s2	NSS	Raj	20
s3	Dancing	Gaurav	13
s4	Sashakt	Bhatia	10
s5	QuickSort	Ms.Sheetal	15
NULL	NULL	NULL	NULL

8 - Find society names in which more than five students have enrolled

```
-- 8
select s.SocName
from society s
join enrollment e
on s.SocID = e.SID
group by s.SocID, s.SocName
having count(e.ROLLNO)>5;
```

Result Grid | Filter Rows:

SocName
Debating

9 - Find the name of the youngest student enrolled in society 'NSS'

```
-- 9
use student_society;
select s.Student_name from Student s
join enrollment e on s.ROLLNO = e.ROLLNO
join society so on so.SocID = e.SID
where so.SocName = 'NSS'
and s.DOB = (select max(s2.DOB) from Student s2
                join enrollment e2 on s2.ROLLNO = e2.ROLLNO
                join society so2 on e2.SID = so2.SocID
                where so2.SocName = 'NSS');
```

Result Grid | Filter Rov

Student_name
Ram

10 - Find the name of the most popular society (on the basis of enrolled students)

```
-- 10
SELECT s.SocName
FROM society s
JOIN enrollment e ON s.SocID = e.SID
GROUP BY s.SocID, s.SocName
ORDER BY COUNT(e.ROLLNO) DESC
LIMIT 1;
```

```
119        order by count(e.ROL
```

Result Grid | Filter Rows:

SocName
Debating

11 - Find the name of two least popular societies (on the basis of enrolled students)

```
-- 11
select s.SocName from society s
join enrollment e on s.SocID = e.SID
group by s.SocID, s.SocName
 order by count(e.ROLLNO) asc
limit 2;
```

Result Grid | Filter Rows:

SocName
QuickSort
NSS

Refresh data re-exe

12 - Find the students names who are not enrolled in any society.

```
-- 12
SELECT s.Student_name
FROM Student s
WHERE NOT EXISTS (
    SELECT 1
    FROM enrollment e
    WHERE e.ROLLNO = s.ROLLNO
);
```

13 - . Find the students names enrolled in at least two societies

```
140     FROM society so
```

```
-- 13
select s.Student_name from Student s
join enrollment e on
e.ROLLNO = s.ROLLNO
group by s.ROLLNO,s.Student_name
having count(distinct e.SID) >=2;
```

Result Grid | Filter Rows:

Student_name
Ram
Raju
Ajay
Aman

14 - Find society names in which maximum students are enrolled

```
-- 14
SELECT so.SocName
FROM society so
JOIN enrollment e ON so.SocID = e.SID
GROUP BY so.SocID, so.SocName
ORDER BY COUNT(e.ROLLNO) DESC
LIMIT 1;
```

Result Grid | Filter Rows

SocName
Debating

15 - Find names of all students who have enrolled in any society and society names in which at least one student has enrolled

```
-- 15
SELECT distinct s.Student_name, so.SocName
FROM Student s
join enrollment e on s.ROLLNO = e.ROLLNO
join society so on e.SID = so.SocID;
```

Result Grid | Filter Rows:

Student_name	SocName
Ram	NSS
Ram	Debating
Raju	QuickSort
Raju	Debating
Ajay	Dancing
Ajay	Debating
Aman	Sashakt
Aman	Debating

16 - Find names of students who are enrolled in any of the three societies ‘Debating’, ‘Dancing’ and ‘Sashakt’.

```
-- 16
select  s.Student_name from student s
join enrollment e on s.ROLLNO = e.ROLLNO
join society so on e.SID = so.SocID
    where   so.SocName  in('Debating' or 'Dancing' and  'Sashakt');
```

Result Grid | Filter Rows:

Student_name
Lakhan
Ram
Ajay
Aman
Raju
Ram
Ajay
Aman
Raju

17 - Find society names such that its mentor has a name with ‘Gupta’ in it.

```
-- 17
select SocName from society
where MentorName like '%Gupta';
```

Result Grid | Filter Rows:

SocName
Debating

18 - Find the society names in which the number of enrolled students is only 10% of its capacity.

```
-- 18
select so.SocName from society so
join enrollment e on so.SocId = e.SID
group by so.SocID,so.SocName, so.TotalSeats
having count(e.ROLLNO) = 0.1 * so.TotalSeats;
```

Result Grid

SocName
Sashakt

19 - Display the vacant seats for each society.

```
-- 19
SELECT
    so.SocName,
    so.TotalSeats - COUNT(e.ROLLNO) AS Vacant_Seats
FROM society so
LEFT JOIN enrollment e
ON so.SocID = e.SID
GROUP BY so.SocID, so.SocName, so.TotalSeats;
```

Result Grid | Filter Rows:

SocName	Vacant_Seats
Debating	19
NSS	19
Dancing	12
Sashakt	9
QuickSort	14

20 - Increment Total Seats of each society by 10%

```sql
select
    TotalSeats + 0.1* Totalseats as newtotalSeats,SocName
    from society;
```

Result Grid | Filter Rows:

newtotalSeats	SocName
27.5	Debating
22.0	NSS
14.3	Dancing
11.0	Sashakt
16.5	QuickSort

21 – Add the enrollment fees paid (‘yes’/’No’) field in the enrollment table.

```sql
-- 21
alter table enrollment
add column feesPaid enum('yes','no');
desc enrollment;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Co

Field	Type	Null	Key	Default	Extra
ROLLNO	char(6)	YES	MUL	NULL	
SID	char(6)	YES	MUL	NULL	
DateofEnrollment	date	YES		NULL	
feesPaid	enum('yes','no')	YES		NULL	

22 - Update date of enrollment of society id ‘s1’ to ‘2018-01-15’, ‘s2’ to the current date and ‘s3’ to ‘2018-01-02’.

```sql
UPDATE enrollment
SET DateofEnrollment = CASE
    WHEN SID = 's1' THEN '2018-01-15'
    WHEN SID = 's2' THEN CURDATE()
    WHEN SID = 's3' THEN '2018-01-02'
END
WHERE SID IN ('s1', 's2', 's3');
```

23 - . Create a view to keep track of society names with the total number of students enrolled in it.

```sql
CREATE VIEW society_enrollment_count AS
SELECT
    so.SocName,
    COUNT(e.ROLLNO) AS Total_Students
FROM society so
LEFT JOIN enrollment e
ON so.SocID = e.SID
GROUP BY so.SocID, so.SocName;
```

Result Grid | Filter Rows:

SocName	Total_Students
Debating	6
NSS	1
Dancing	1
Sashakt	1
QuickSort	1

24 - Find student names enrolled in all the societies.

```sql
-- 24
SELECT s.Student_name
FROM Student s
JOIN enrollment e ON s.ROLLNO = e.ROLLNO
GROUP BY s.ROLLNO, s.Student_name
HAVING COUNT(DISTINCT e.SID) = (
    SELECT COUNT(*) FROM society
);
```

25 - . Count the number of societies with more than 5 students enrolled in it

```sql
-- 25
SELECT COUNT(*) AS No_of_Societies
FROM (
    SELECT e.SID
    FROM enrollment e
    GROUP BY e.SID
    HAVING COUNT(e.ROLLNO) > 5
) AS temp;
```

Result Grid | Filter R

	No_of_Societies
▶	1

26 - Add column Mobile number in student table with default value '9999999999'

```sql
-- 26
alter table Student
add column MobileNo varchar(15) default '99999999' ;
```

27 - Find the total number of students whose age is > 20 years.

```sql
-- 27
SELECT COUNT(*) AS Total_Students
FROM Student
WHERE TIMESTAMPDIFF(YEAR, DOB, CURDATE()) > 20;
```

```sql
237     FROM Student s
```

Result Grid | Filter Row

	Total_Students
▶	3

28 - . Find names of students who were born in 2001 and are enrolled in at least one society.

```sql
-- 28
SELECT DISTINCT s.Student_name
FROM Student s
JOIN enrollment e
ON s.ROLLNO = e.ROLLNO
WHERE YEAR(s.DOB) = 2001;
```

Result Grid | Filter

	Student_name
▶	Raju

29 - . Count all societies whose name starts with 'S' and ends with 't' and at least 5 students are enrolled in the society.

```sql
-- 29
SELECT COUNT(*)
FROM society
WHERE SocName LIKE 'S%t'
AND SocName IN (
    SELECT SocName
    FROM enrollment
    GROUP BY SocName
    HAVING COUNT(ROLLNO) >= 5
);
```

Result Grid | Filter

	COUNT(*)
▶	1

30 - Display the following information: Society name, Mentor name ,Total Capacity ,Total Enrolled ,Unfilled Seats.

```sql
-- 30
SELECT
    so.SocName,
    so.MentorName,
    so.TotalSeats AS Total_Capacity,
    COUNT(e.ROLLNO) AS Total_Enrolled,
    so.TotalSeats - COUNT(e.ROLLNO) AS Unfilled_Seats
FROM society so
LEFT JOIN enrollment e
ON so.SocID = e.SID
GROUP BY so.SocID, so.SocName, so.MentorName, so.TotalSeats;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

SocName	MentorName	Total_Capacity	Total_Enrolled	Unfilled_Seats
Debating	Aman Gupta	25	6	19
NSS	Raj	20	1	19
Dancing	Gaurav	13	1	12
Sashakt	Bhatia	10	1	9
QuickSort	Ms.Sheetal	15	1	14